MOTOSECURE | A NEXUS TECH LABS PRODUCT

System Architecture Document

Urban motorcycle-taxi regulation and security platform for municipalities in Cameroon

Document ID	MS-ARCH-001
Version	1.0 (draft for review)
Date	2 November 2026
Status	Draft for institutional review
Classification	Confidential
Publisher	NEXUS Tech Labs
Baseline	MotoSecure source archive as reviewed on 2 November 2026 (backend, web, mobile)
Approval	[Name, title and date - to be completed by the document owner]

This document describes the software as implemented in the reviewed source archive. Statements about hosting and deployment reflect the owner's reported configuration and were not independently verified. It is not a security certification, penetration-test report or compliance assessment.

Contents

1. Introduction 3
1.1 Purpose and scope 3
1.2 Basis of this document 3
1.3 Glossary 3
2. System overview 3
2.1 Stakeholders and actors 3
2.2 Architecture at a glance 4
2.3 Technology stack 4
3. Components 5
3.1 API (backend) 5
3.2 Web application 5
3.3 Mobile application (enforcement) 5
3.4 Data store 6
4. Data model 6
5. Multi-tenancy and authorization architecture 6
6. Authentication and session management 7
7. Key workflows and data flows 7
7.1 Syndicate enrollment and approval 7
7.2 Rider enrolment and badge issuance 7
7.3 Badge verification 8
7.4 Offline operation 9
7.5 Incident reporting 9
7.6 Audit logging 9
8. Deployment architecture 9
8.1 Environments (as reported by the owner) 9
8.2 Configuration and secrets 10
8.3 Build and release 10
9. Security boundaries and trust model 10
10. External services and integrations 11
11. Non-functional characteristics 11
12. Known limitations and recommended architecture work 11
Appendix A. API route inventory 12
Appendix B. Repository layout 13

1. Introduction

1.1 Purpose and scope

This document describes the architecture of MotoSecure as implemented in the reviewed source archive: its components, deployment, data flows, interfaces, authentication model and security boundaries. It is written for technical reviewers at government institutions and prospective pilot partners.

MotoSecure lets a municipality register motorcycle-taxi syndicates, riders and motorcycles; issue each rider a QR badge; and let enforcement officers check a badge on a phone, online or offline. It is multi-tenant: every municipality's data is isolated from every other's.

1.2 Basis of this document

Every statement is derived from the source code, the automated tests and the static-analysis output in the repository. Where something is reported by the owner rather than visible in the code (for example the hosting providers), it is marked as reported. Features that do not exist are listed explicitly in section 12 and in the Software Requirements Specification rather than implied.

> **How to read verification claims**
> Evidence is described as **executed** (run during the review, results recorded), **written** (a test exists but could not be run in the review environment) or **reviewed** (inspected by reading only). The review environment had no network access, so the HTTP-level integration tests could not be run there.

1.3 Glossary

Term	Meaning in MotoSecure
Tenant / municipality	A commune or council. Every record belongs to exactly one tenant.
Syndicate	A motorcycle-taxi union operating in a zone of a municipality. Registered by its administrator and approved by the council.
Rider	An individual driver enrolled under one syndicate.
Badge	A server-generated identifier (MS- plus 12 hexadecimal characters) encoded as a QR code and carried by the rider.
Verdict	The result of a check: valid, expired, suspended, flagged or not_found.
Snapshot	A compact copy of a municipality's riders cached on an officer's phone for offline checks.
Council roles	super_admin, mayor and municipal_staff.

2. System overview

2.1 Stakeholders and actors

Role	Scope	Purpose in the system
super_admin	Platform operator (NEXUS). Cross-municipality; optional ?tenant= scoping.	Creates municipalities; full read/write across all; no tenant required on the account.
mayor	One municipality.	Command dashboard; approves syndicates; changes rider status; edits own municipality's contact details; reads audit log.

Role	Scope	Purpose in the system
municipal_staff	One municipality.	Same operational rights as the mayor except editing the municipality record.
syndicate_admin	One syndicate within one municipality.	Manages own syndicate's contact details, riders and motorcycles (once the syndicate is approved). Cannot approve anything.
enforcement_officer	One municipality.	Mobile verification, incident reporting and offline sync. Reduced view of riders (no phone, national ID or date of birth).

2.2 Architecture at a glance

The system has three client surfaces sharing one API and one database. There is no server-side rendering and no background worker: all logic runs in the API process, and the only outbound integration is email for password reset.

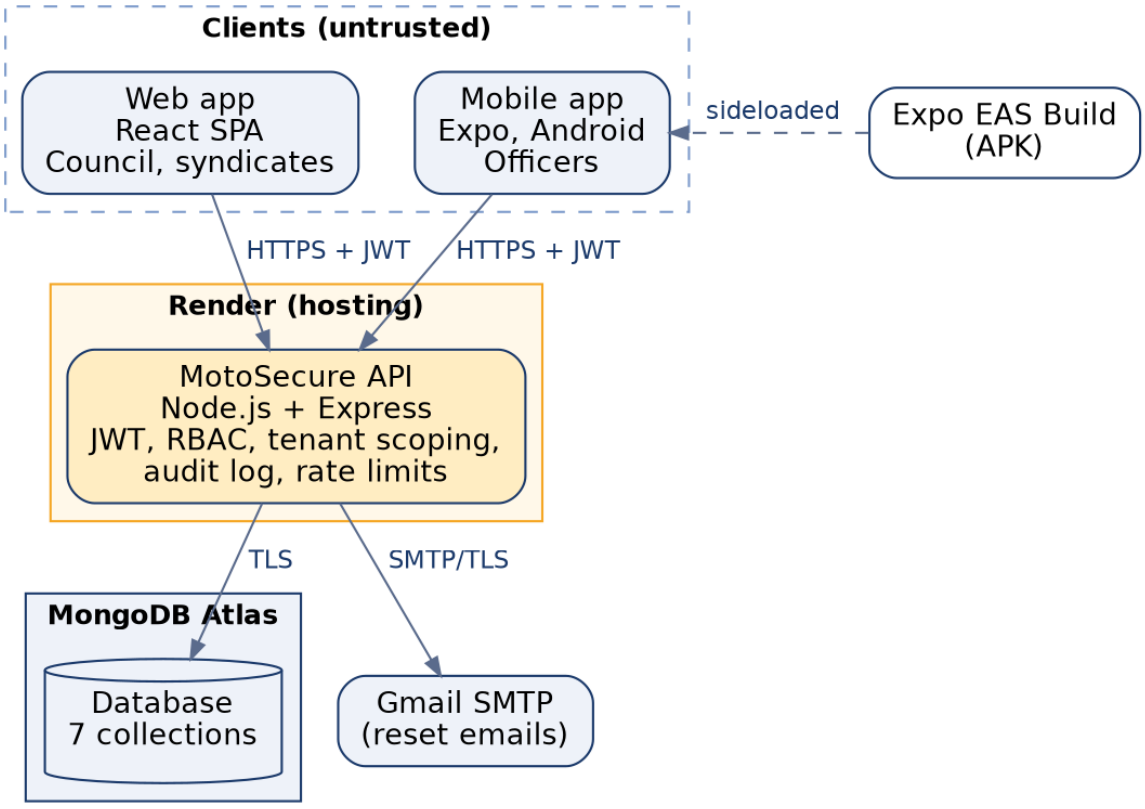


Figure 1 - Components, hosting and trust boundaries (as reported by the owner)

2.3 Technology stack

Layer	Technology	Main libraries (versions from the package manifests)
Runtime	Node.js (engines: >= 18)	Express ^4.19.2, Mongoose ^8.5.2, jsonwebtoken ^9.0.2, bcryptjs ^2.4.3, helmet ^7.1.0, cors ^2.8.5, express-rate-limit ^7.4.0, qrcode ^1.5.4, nodemailer ^6.9.14, morgan ^1.10.0
Web	React ^18.3.1 + Vite ^5.4	Tailwind CSS ^3.4, React Router ^6.26, react-i18next ^15, Recharts ^2.12, axios, framer-motion

Layer	Technology	Main libraries (versions from the package manifests)
Mobile	Expo / React Native (Android)	Manifest in the reviewed archive: Expo ~51.0.28, React Native 0.74.5, expo-camera ~15, AsyncStorage 1.23, React Navigation 6. The owner reports a local upgrade to Expo SDK 57 that is not reflected in the archive.
Database	MongoDB (Atlas)	Accessed only through Mongoose models
Tests	node:test (built in)	supertest ^7 and mongodb-memory-server ^10 (dev dependencies, integration suite)

3. Components

3.1 API (backend)

A single Express application. Request handling passes through the same ordered layers on every route:

1. **Transport hardening** (app.js): security headers (helmet), CORS allow-list, 100 KB JSON limit, rejection of object keys beginning with $ or containing a dot or prototype names, global rate limit of 300 requests per minute per client.
2. **Authentication** (middleware/auth.js): validates the Bearer token with an explicit HS256 algorithm list, reloads the user from the database on every request, rejects deactivated users and tokens issued before the last password change.
3. **Authorization** (authorize middleware plus lib/policy.js): role checks, then database filters built by pure functions that pin every query to the caller's municipality (and syndicate for a syndicate_admin).
4. **Input handling**: field allow-lists decide which properties a role may write; identifiers, enumerations and lengths are validated before any database call.
5. **Business logic and persistence**: Mongoose models with indexes and schema validation.
6. **Audit and error handling**: administrative actions are appended to the audit log; a central handler maps errors to safe responses and hides internals in production.

Supporting modules: lib/verdict.js (verification rules), lib/enrollment.js (which syndicate a new rider or bike belongs to), lib/audit.js, lib/sendEmail.js and lib/config.js (startup checks). server.js refuses to start without MONGO_URI and a JWT secret of at least 32 characters.

3.2 Web application

- **Public pages:** landing page, sign-in, syndicate enrollment (municipality chosen from a list of real councils), forgot and reset password.
- **Signed-in shell (responsive, light and dark):** dashboard with scoped totals and a rider-status chart; syndicates (council can approve, reject or suspend); riders (list and badge QR view); bikes; enforcement log; audit log (council roles only).
- **Not present:** forms to enrol riders or motorcycles (the API supports it), user management, municipality onboarding screens. See section 12.

3.3 Mobile application (enforcement)

- **Screens:** sign-in, home (recent scans, pending-sync indicator), camera QR scan, manual badge entry, result, incident report, settings (language, theme, sign-out).
- **Offline design:** the app downloads a municipality snapshot when online and falls back to it on connectivity failure (no response, 12-second timeout, or HTTP 502-504). Results from the snapshot are labelled with the snapshot date and queued for later upload. See section 7.4.
- **Local storage:** session token, last known profile, snapshot, queue and recent scans are held in the device's application storage (AsyncStorage). Signing out removes them.

3.4 Data store

MongoDB holds seven collections, listed in section 4. Uniqueness and tenancy are enforced by indexes: email (User), badgeId (Rider), tenant+name (Syndicate), tenant+plateNumber (Bike), officer+clientScanId (EnforcementLog, for offline idempotency).

4. Data model

Collection	Principal fields	Notes
Tenant	name, slug (unique), region, mayorName, contact details, plan (trial/standard/premium), status (active/pending/suspended), settings	Municipality; root of isolation. Settings are stored but not enforced.
User	tenant, syndicate, fullName, email (unique), role, passwordHash, isActive, failedLoginCount, lockUntil, passwordChangedAt, reset-token hash and expiry	bcrypt hash never returned by the API; reset token stored only as a SHA-256 hash.
Syndicate	tenant, name (unique per tenant), zone, president name/phone, contactEmail, status (pending/approved/rejected/suspended), trustScore, notes, documents[] (URLs)	memberCount exists but is not maintained; trustScore is a manually set number.
Rider	tenant, syndicate, badgeId (unique, immutable), fullName, phone, licenseNumber, licenseExpiresAt, status (active/suspended/under_review/revoked), bike, counters; dateOfBirth and nationalIdNumber	dateOfBirth and nationalIdNumber are excluded from every query unless explicitly selected.
Bike	tenant, syndicate, rider, plateNumber (unique per tenant), make/model/colour/year, chassisNumber, insurance and technical-control dates, roadworthy, status	Linked to a rider of the same syndicate only.
EnforcementLog	tenant, rider, bike, officer, type (verification/incident/warning/impound), result, source (live/offline_sync), clientScanId, location, description, severity	clientScanId + officer is unique, making offline sync idempotent.
AuditLog	tenant, actor, actorName, actorRole, action, targetType/Id/Label, metadata, ip	Append-only through the application layer; no update/delete routes exist.

Personal data held: account holders (name, email, phone); riders (name, phone, licence number and expiry, photo URL, and optionally date of birth and national ID number); enforcement observations (rider, officer, free-text description, location label); audit entries (actor name, role, IP address).

5. Multi-tenancy and authorization architecture

Isolation is enforced in the API, not in the database: all tenants share one database and every document carries a tenant reference. The rules live in lib/policy.js as pure functions so they can be tested without a database.

- **tenantScope(user, query)** returns { tenant: <the caller's municipality> }. A query-string tenant is honoured only for super_admin. A user with no resolvable municipality receives a filter that matches nothing (an ObjectId no document has), never an empty filter.
- **syndicateDataScope** adds the caller's syndicate for a syndicate_admin. **syndicateRecordFilter** lets a syndicate_admin address only their own syndicate by id.
- **Write allow-lists** (riderWritableFields, bikeWritableFields, syndicateWritableFields, tenantWritableFields) define the only properties each role may set. Status, badge id, counters, tenant and syndicate are never client-assignable through generic create or edit routes.

- **Read projections:** nationalIdNumber and dateOfBirth are excluded from queries by default; officers additionally never receive phone numbers.
- **Verification views:** verificationView decides between the full and the limited view of a rider (section 7.3).

> **Evidence**
> These functions are covered by 20 unit tests that were **executed** during the review (all passing). The route wiring that applies them is covered by 122 HTTP-level integration tests that are **written but were not executed** (see the Security Assessment Report).

6. Authentication and session management

- **Credentials:** email and password. Passwords are hashed with bcrypt (cost 12 by default). Registration and reset require at least 8 characters, a letter and a digit, and at most 72 bytes.
- **Tokens:** JWT signed with HS256, default lifetime 7 days, sent as a Bearer header. The token carries the user id; role and municipality are always re-read from the database. There are no refresh tokens and no server-side revocation list: a session ends at expiry, when the account is deactivated, or when the password changes.
- **Brute-force controls:** five consecutive failures lock the account for 15 minutes (audited); login is also limited to 30 attempts per 15 minutes per client address. Failure responses are identical for unknown accounts, wrong passwords and locked accounts, and a dummy password comparison equalises timing.
- **Password reset:** a random 256-bit token is emailed as a link valid for one hour and stored only as a SHA-256 hash. The request endpoint answers identically whether or not the account exists. A successful reset invalidates earlier sessions and clears any lock.
- **Client storage:** the web app keeps the token in browser localStorage; the mobile app keeps it in AsyncStorage. Both are exposed to script injection or device compromise respectively; see section 12.

7. Key workflows and data flows

7.1 Syndicate enrollment and approval

1. A syndicate representative opens the public enrollment page, picks a municipality from the list and submits union details, their own details and a password.
2. The API creates a syndicate with status pending and a syndicate_admin account, and writes a syndicate.registered audit entry. The new admin can sign in but cannot enrol riders or bikes yet.
3. A council user reviews the syndicate and approves, rejects or suspends it. The change, the previous and new status and any notes are recorded in the audit log.

7.2 Rider enrolment and badge issuance

1. A syndicate_admin (own, approved syndicate) or council user creates a rider. The server sets status to under_review and generates the badge id; client-supplied status, badge id, counters, tenant and syndicate are ignored.
2. A bike may be registered and linked to a rider of the same syndicate; the link is validated on the server.
3. A council user activates the rider through the audited status route. A manager can then generate the QR image of the badge for printing.

7.3 Badge verification

The scanner sends the badge id to a single endpoint. What it returns depends on who is asking:

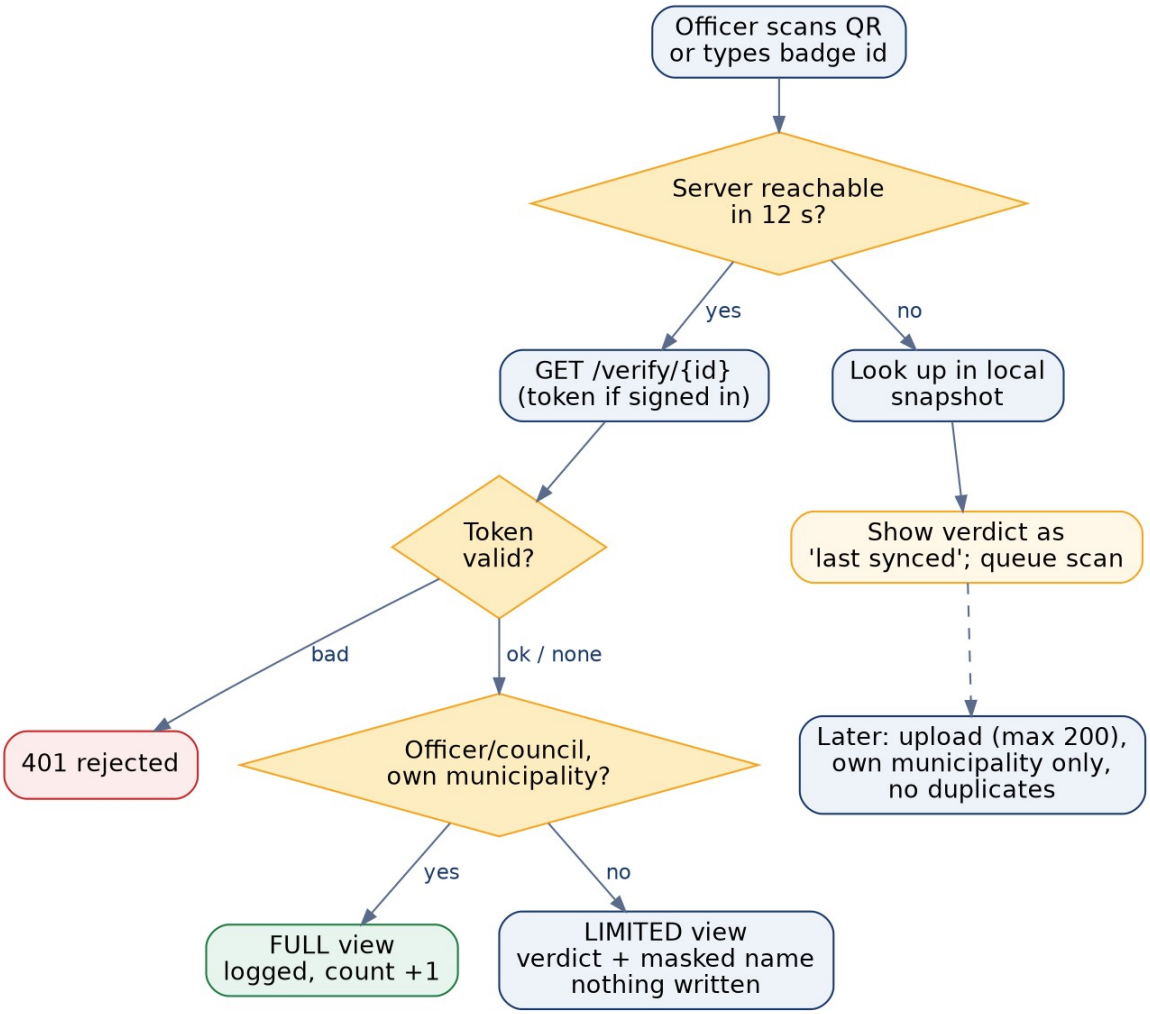


Figure 2 - Verification decision flow, online and offline

Caller	Result	Side effects
Officer, mayor or staff, rider in their own municipality; super_admin	Full view: name, photo URL, syndicate, bike, licence expiry, verdict	Scan logged with officer and time; rider's verification count incremented
Officer from another municipality; syndicate_admin	Limited view: verdict and masked name (for example "Enow P.")	None
Anonymous caller	Limited view	None; limited to 30 requests per minute per client address
Caller with an invalid or expired token	HTTP 401	None; never treated as anonymous

Verdict rules: suspended riders are suspended; revoked or under-review riders are flagged; an expired licence is expired; otherwise valid; an unknown badge is not_found. Insurance and roadworthiness data are shown but do not change the verdict, and a suspended syndicate does not change its riders' verdicts (open decisions in the SRS).

Design limitation: static QR codes

The QR code encodes only the badge id. A photograph of a badge can be copied onto another person. The system mitigates this by showing the rider's name, syndicate and plate number to the officer for comparison, and by making revocation effective on the next scan. Rotating or cryptographically signed QR codes are not implemented.

7.4 Offline operation

1. While online (on opening the home screen) the app downloads the municipality snapshot: badge id, name, status, licence expiry, syndicate name and plate. It contains no phone numbers, national ID numbers or dates of birth.
2. When a check cannot reach the server it uses the snapshot, shows the verdict with the snapshot date, and queues the scan with a unique scan id. Authentication or other server errors are shown as errors and never answered from the cache.
3. When online again the queue is uploaded in batches of at most 200. The server accepts only riders of the officer's own municipality and valid verdicts, rejects scans older than 30 days, clamps future timestamps, and ignores duplicate scan ids, so a retried upload cannot create duplicates.
4. A revocation made after the last snapshot refresh is not visible to an offline device until it next connects. Offline results are therefore indicative.

7.5 Incident reporting

An officer or council user records an incident, warning or impound against a rider of their own municipality with a severity from low to critical and an optional description of up to 1,000 characters. The rider's incident count is incremented and the entry appears in the enforcement log. Reporting requires connectivity.

7.6 Audit logging

The following actions write an audit entry: syndicate registered, created, updated, approved, rejected and suspended; rider created, updated and status changed; bike created and updated; municipality created and updated; account locked; password reset completed. Entries hold actor, role, action, target label, before and after values where relevant, IP address and time, and never passwords or tokens. The log is append-only through the application; read access is limited to council roles within their own municipality.

8. Deployment architecture

8.1 Environments (as reported by the owner)

Item	Reported configuration	Status
API	Render web service, root directory backend, npm install and npm start; configuration through the Render dashboard	**Reported; not verified**
Web	Static build (npm run build, publish dist) served by Render	**Reported; not verified**
Database	MongoDB Atlas cluster	**Reported; not verified**
Mobile	Android APK built with Expo EAS (preview profile, internal distribution) and installed by sideloading	**Reported; not verified**
Tier	Entry-level / free tiers of the above services; the owner reports a first request after idle taking roughly 30-50 seconds	**Reported; not verified**
Environments	A single database is used for demonstration; separate development, staging and production databases are not defined in the repository	**Open**

8.2 Configuration and secrets

All configuration is through environment variables; no secret is committed (.env is git-ignored and a .env.example template is provided). The variables are listed below; values are intentionally omitted.

Variable	Component	Requirement	Notes
MONGO_URI	API	Required	MongoDB connection string. Use a separate database per environment.
JWT_SECRET	API	Required	At least 32 characters; startup fails otherwise.
JWT_EXPIRES_IN	API	Optional (7d)	Token lifetime.
NODE_ENV	API	Recommended	production hides error details and email links in logs.
CLIENT_ORIGINS	API	Required in practice	Comma-separated browser origin allow-list; no wildcard support.
TRUST_PROXY	API	Optional (1)	Reverse-proxy hops, so rate limits see the real client IP.
BCRYPT_ROUNDS	API	Optional (12)	Password hashing cost.
EMAIL_USER, EMAIL_APP_PASSWORD	API	For password reset	Gmail SMTP account and app password.
FRONTEND_URL	API	For password reset	Base URL placed in reset links (should be https).
VITE_API_URL	Web build	Required	Public API base URL.
VITE_DEMO_ACCOUNTS	Web build	Optional (empty)	Build-time list of demo logins; leave empty for real deployments.
SEED_SUPERADMIN_EMAIL / _PASSWORD	Seed script	Demo only	Seeding also requires the --confirm-wipe flag.

8.3 Build and release

The repository contains no continuous-integration workflow, container definition or infrastructure-as-code. Deployment is by pushing to GitHub, after which the hosting platform rebuilds. The API is stateless apart from the database, so it could be run as several instances behind a load balancer, but this has not been tested.

9. Security boundaries and trust model

Boundary	What crosses it	Controls in the implementation
Internet to API	All client requests	TLS terminated by the hosting platform (assumed); CORS allow-list; rate limits; 100 KB body limit; operator-key rejection; authentication and authorization on every non-public route
Client devices	Session token, cached snapshot (mobile)	Tokens in localStorage / AsyncStorage (not hardware-backed); snapshot excludes contact and identity numbers; wiped on sign-out
API to database	All persistent data	Connection string in environment; Mongoose schema validation; tenant filters on every query; Atlas network and encryption settings are outside the repository

Boundary	What crosses it	Controls in the implementation
API to email provider	Password-reset links	Sent over SMTP with an app password; links contain a one-time token valid for one hour
Administrators of hosting accounts	Full access to configuration and data	Outside the application; governed by the Security, Privacy and Governance Policy

10. External services and integrations

Service	Used for	Data shared
Render	Hosting of API and web build	All application traffic and configuration
MongoDB Atlas	Database	All stored data
Gmail SMTP	Password-reset email	Recipient email address and reset link
Expo EAS	Building the Android app	Source code and build configuration (no production data)
Google Fonts	Web typography (loaded by index.html)	Visitor IP address and browser details are sent to Google when a page loads

11. Non-functional characteristics

- **Scalability:** list endpoints return at most 1,000 records by default (2,000 maximum) and there is no pagination or search; offline snapshots and offline sync are sized for a municipality, not a nation.
- **Availability:** a single API instance on a hosting tier that sleeps when idle; no availability target has been defined or measured.
- **Performance:** no load or performance testing was performed.
- **Observability:** request logs only (not in the test environment); no central monitoring, metrics or alerting.
- **Recoverability:** no backup or restore procedure is defined in the repository.

12. Known limitations and recommended architecture work

Area	Current state	Recommendation
Web session storage	Token in localStorage	Move to an httpOnly, secure, same-site cookie with CSRF protection, or shorten token life and add refresh tokens
Mobile storage	Token and snapshot in AsyncStorage	Use hardware-backed secure storage for the token and encrypt the snapshot
Token revocation	Expiry, deactivation or password change only	Add refresh tokens and a revocation mechanism; consider a shorter lifetime for officers' devices
User administration	No API or screen to create officers, deactivate users or change one's own password	Add managed user provisioning with deactivation and a change-password endpoint

Area	Current state	Recommendation
Multi-factor authentication	Not implemented	Require it for super_admin, mayor and municipal_staff
QR authenticity	Static identifier	Rotating or signed QR codes; photo comparison in the app
Rider and bike enrolment UI	API only	Add web forms so syndicates can enrol riders and bikes without tooling
Audit immutability	Application-level only	Use a database role without update/delete rights on the audit collection and ship entries to external storage
Environments and operations	Single demo database, no CI, backups or monitoring	Separate environments, CI running the test suite, automated backups with restore tests, uptime and error monitoring
Data residency	Provider regions not documented	Confirm hosting regions against institutional and legal requirements (see the Governance Policy)

Appendix A. API route inventory

Generated from the source by scripts/static-checks.js (31 routes). Access reflects the middleware declared on each route; "scoped to the caller" means the result is filtered by the rules in section 5.

Method	Path	Access	Purpose
GET	`/api/audit-logs`	mayor, municipal_staff, super_admin	Read the audit trail
POST	`/api/auth/login`	Public (no sign-in)	Sign in; returns JWT
POST	`/api/auth/register-syndicate`	Public (no sign-in)	Self-service syndicate enrollment (creates pending syndicate + admin)
POST	`/api/auth/forgot-password`	Public (no sign-in)	Request password-reset email
POST	`/api/auth/reset-password`	Public (no sign-in)	Complete password reset
GET	`/api/auth/me`	Any signed-in role, results scoped to the caller	Current user and municipality
GET	`/api/bikes`	Any signed-in role, results scoped to the caller	List motorcycles
POST	`/api/bikes`	syndicate_admin, municipal_staff, mayor, super_admin	Register a motorcycle
PATCH	`/api/bikes/:id`	syndicate_admin, municipal_staff, mayor, super_admin	Edit a motorcycle
GET	`/api/enforcement/verify/:badgeId`	Optional sign-in (see 7.3)	Verify a badge (limited or full view)
POST	`/api/enforcement/sync-scans`	enforcement_officer, municipal_staff, mayor, super_admin	Upload scans recorded offline
POST	`/api/enforcement/incident`	enforcement_officer, municipal_staff, mayor, super_admin	Record incident / warning / impound
GET	`/api/enforcement/logs`	Any signed-in role, results scoped to the caller	Verification and incident timeline
GET	`/api/enforcement/stats`	Any signed-in role, results scoped to the caller	Dashboard roll-up counts

Method	Path	Access	Purpose
GET	`/api/riders/offline-snapshot`	enforcement_officer, municipal_staff, mayor, super_admin	Compact rider list cached by the mobile app
GET	`/api/riders`	Any signed-in role, results scoped to the caller	List riders (no national ID / date of birth)
GET	`/api/riders/:id/badge-qr`	syndicate_admin, municipal_staff, mayor, super_admin	Generate QR image of the badge id
GET	`/api/riders/:id`	Any signed-in role, results scoped to the caller	Read one rider
POST	`/api/riders`	syndicate_admin, municipal_staff, mayor, super_admin	Enroll a rider (always starts under review)
PATCH	`/api/riders/:id`	syndicate_admin, municipal_staff, mayor, super_admin	Edit rider details (not status/badge/tenant)
PATCH	`/api/riders/:id/status`	municipal_staff, mayor, super_admin	Change rider status (audited)
GET	`/api/syndicates`	Any signed-in role, results scoped to the caller	List syndicates (syndicate_admin: own only)
GET	`/api/syndicates/:id`	Any signed-in role, results scoped to the caller	Read one syndicate
POST	`/api/syndicates`	municipal_staff, mayor, super_admin	Council creates a syndicate
PATCH	`/api/syndicates/:id/status`	municipal_staff, mayor, super_admin	Approve / reject / suspend (audited)
PATCH	`/api/syndicates/:id`	municipal_staff, mayor, super_admin, syndicate_admin	Edit contact details (non-status fields)
GET	`/api/tenants`	Any signed-in role, results scoped to the caller	List municipalities (own only, except super_admin)
GET	`/api/tenants/public`	Public (no sign-in)	Names of municipalities open to enrollment
GET	`/api/tenants/lookup/:slug`	Public (no sign-in)	Minimal public lookup by code
POST	`/api/tenants`	super_admin	Create municipality
PATCH	`/api/tenants/:id`	super_admin, mayor	Edit municipality (mayor: contact fields only)

Appendix B. Repository layout

Path	Contents
backend/app.js, server.js	Application assembly; startup, configuration checks and listening
backend/routes, models, middleware, lib	Endpoints; data models; authentication, security and error handling; policy, validation, verdict, audit and email helpers
backend/seed	Demonstration data generator (requires --confirm-wipe)
backend/tests	unit (pure logic and real middleware with stubs) and integration (HTTP + database) suites; run.js runner
web/src	React application: pages, components, contexts, i18n dictionaries
mobile/src	Expo application: screens, navigation, offline cache, i18n

Path	Contents
scripts/static-checks.js	Repeatable source scanner and route inventory generator
docs	These documents, build scripts and recorded evidence